



UNIVERSITÀ
DEGLI STUDI
FIRENZE

IMDB graph

Francesca Lanzillotta



- 1 Building The Graph
- 2 Most prolific actors
- 3 Diameter
- 4 Largest Shared Cast
- 5 Building The Actors' Graph

Goal

From the given `.tsv` file, build a *Bipartite Unweighted Graph* where nodes are actors/actresses and movies.

Goal

From the given `.tsv` file, build a *Bipartite Unweighted Graph* where nodes are actors/actresses and movies.

There is an edge between an actor/actress node *A* and movie node *M* if *A* participated in *M*.

Nodes

Each node in the Graph contains specific attributes:

Nodes

Each node in the Graph contains specific attributes:

- **Id**: to uniquely identify actors and movies within the graph

Nodes

Each node in the Graph contains specific attributes:

- **Id**: to uniquely identify actors and movies within the graph
- **Name**: entity name extracted from file

Nodes

Each node in the Graph contains specific attributes:

- **Id**: to uniquely identify actors and movies within the graph
- **Name**: entity name extracted from file
- **Type**: actor or movie

Nodes

Each node in the Graph contains specific attributes:

- **Id**: to uniquely identify actors and movies within the graph
- **Name**: entity name extracted from file
- **Type**: actor or movie
- **Year** (*only movie nodes*): indicates the release year



Supporting Structures

The algorithm uses two dictionaries as supporting structures:

Supporting Structures

The algorithm uses two dictionaries as supporting structures:

- Actor dictionary

```
{actor_name : actor_id}
```

- Movie dictionary

```
{movie_name : movie_id}
```

```
1 with open("imdb-actors-actresses-movies.tsv") as file:
2
3     for line in file:
4
5         line = line.split("\t")
6         # line contains array [actor, movie]
7
8         if line[0] not in actor_dict.keys():
9             # create actor node
10            ...
11        else:        # retrieve actor's id
12            actor_id = actor_dict[line[0]]
13
14        if line[1] not in movie_dict.keys():
15            # create movie node
16            ...
17        else:        # retrieve movie's id
18            movie_id = movie_dict[line[1]]
19
20        imdb_graph.add_edge(actor_id, movie_id)
```

Resulting Graph

The algorithm proposed created a graph with:

- 3 110 737 nodes

Resulting Graph

The algorithm proposed created a graph with:

- 3 110 737 nodes
 - 2 364 796 actor nodes
 - 745 941 movie nodes

Resulting Graph

The algorithm proposed created a graph with:

- 3 110 737 nodes
 - 2364 796 actor nodes
 - 745 941 movie nodes
- 8 103 960 edges

Goal

Find the actor who did more movies, considering only the movies up to year x with x in $\{1930, 1940, 1950, 1960, 1970, 1980, 1990, 2000, 2010, 2020\}$.

Algorithm

The algorithm examines every actor one by one and only once.

For each one, it counts the movies made considering an increasing year range, and stores the results in a dictionary

```
{year: number of movies}
```

Algorithm

The algorithm examines every actor one by one and only once.

For each one, it counts the movies made considering an increasing year range, and stores the results in a dictionary

```
{year: number of movies}
```

The cumulative results are stored in a different dictionary

```
{year: (number of movies, actor_id)}
```

```
1 for actor_node in actor_dict.values():
2     actor_counter = {}
3
4     # initialize actor counter dict
5     for year in year_range:
6         actor_counter[year] = 0
7
8     # check every movie the actor participated in
9     for movie_node in graph.adj[actor_node]:
10         movie_year = graph.nodes[movie_node]["year"]
11         # skip movies with no year
12         if movie_year is not None:
13             for year in year_range:
14                 if movie_year <= year:
15                     actor_counter[year] += 1
16
17     # find max comparing all dictionaries entries
18     for year in year_range:
19         if actor_counter[year] > total_max[year][0]:
20             total_max[year] = (actor_counter[year], actor_node)
```

Results

Year	Actor	Movies
2020	Byron, Tom (I)	1882
2010	Byron, Tom (I)	1824
2000	Byron, Tom (I)	1610
1990	Blanc, Mel	1024
1980	Blanc, Mel	989
1970	Blanc, Mel	946
1960	Onoe, Matsunosuke	925
1950	Onoe, Matsunosuke	925
1940	Onoe, Matsunosuke	925
1930	Onoe, Matsunosuke	925

Goal

Restricting to the largest connected component of the graph, compute exactly the diameter of the graph, considering only the movies up to year x with x in $\{1930, 1940, 1950, 1960, 1970, 1980, 1990, 2000, 2010, 2020\}$

To consider only the movies released up to the years specified in the year range

```
1 year_range = {1930, 1940, 1950, 1960, 1970, 1980, 1990,
2               2000, 2010, 2020}
3 for year in year_range:
4
5     # create subgraph with movies released up to year
6     subgraph = subgraph_by_year(imdb_graph, year)
7
8     d = bounded_diameter(subgraph)
```

Algorithm

The algorithm implemented performs Breadth First Searches on the nodes in a specific order inferred by some properties of the BFS-trees, while at the same time:

Algorithm

The algorithm implemented performs Breadth First Searches on the nodes in a specific order inferred by some properties of the BFS-trees, while at the same time:

- refining a lower bound (i.e. the maximum eccentricity found)
- upper bounding the remaining nodes' eccentricities

Algorithm

The algorithm implemented performs Breadth First Searches on the nodes in a specific order inferred by some properties of the BFS-trees, while at the same time:

- refining a lower bound (i.e. the maximum eccentricity found)
- upper bounding the remaining nodes' eccentricities

The algorithm stops when the lower bound is bigger than the upper bound.

Algorithm

- Constructs a BFS-Tree starting from a high degree node *source*

Algorithm

- Constructs a BFS-Tree starting from a high degree node *source*
- Considering the BFS-Tree levels one at a time in a bottom-up fashion, compute the eccentricities of all nodes in the level

Algorithm

- Constructs a BFS-Tree starting from a high degree node *source*
- Considering the BFS-Tree levels one at a time in a bottom-up fashion, compute the eccentricities of all nodes in the level
- If in a generic level i , the maximum eccentricity found M is greater then $2 * (i - 1)$, the algorithm can stop.

Algorithm

- Constructs a BFS-Tree starting from a high degree node *source*
- Considering the BFS-Tree levels one at a time in a bottom-up fashion, compute the eccentricities of all nodes in the level
- If in a generic level i , the maximum eccentricity found M is greater then $2 * (i - 1)$, the algorithm can stop.

It can be proved that all the eccentricities of the nodes in the upper layers of the tree are bounded by $\max\{M, 2 \cdot (i - 1)\}$

```
1 while ub > lb:
2
3     F_i = layers[i]
4
5     # compute B_i(source)
6     B_i = 0
7     for node in F_i:
8         ecc_i = max(nx.single_source_shortest_path_length
9                     (largest_cc, node).values())
10        if ecc_i > B_i:
11            B_i = ecc_i
12
13    if max(lb, B_i) > 2 * (i - 1): # lb > ub
14        return max(lb, B_i)
15    else:
16        lb = max(lb, B_i)
17        ub = 2 * (i - 1)
18
19    i -= 1
20
21 return lb
```

Results

Year	Diameter	BFS Performed
2020	32	47
2010	30	132
2000	28	41
1990	30	6
1980	25	50
1970	25	52
1960	34	12
1950	38	75
1940	36	66
1930	32	561

Goal

Which is the pair of movies that share the largest number of actors?

Algorithm

The algorithm checks for each movie pair combination possible the number of common elements in their cast (*set intersection*).

Algorithm

The algorithm checks for each movie pair combination possible the number of common elements in their cast (*set intersection*).

The total number of possible movie pair combinations is

$$\binom{n}{2} = \frac{n!}{2 \cdot (n-2)!} = \frac{n(n-1)}{2}$$

where n is the number of movies.

Algorithm

The algorithm checks for each movie pair combination possible the number of common elements in their cast (*set intersection*).

The total number of possible movie pair combinations is

$$\binom{n}{2} = \frac{n!}{2 \cdot (n-2)!} = \frac{n(n-1)}{2}$$

where n is the number of movies.

The total number of intersections can be much lower.



Algorithm

The current maximum provides a lower bound for the sought value

Algorithm

The current maximum provides a lower bound for the sought value

- all the movies with a cast smaller than the current lower bound can be discarded

```
1 # check movies in order
2 for i in range(len(movie_list) - 1):
3
4     if movie_list[i] not in discarded:
5
6         cast = set(imdb_graph.adj[movie_list[i]])
7
8         if len(cast) > max_shared_cast:
9
10            # check the remaining movies
11            for j in range(i+1, len(movie_list)):
12
13                if movie_list[j] not in discarded:
14                    other_cast = set(imdb_graph.adj[movie_list[j]])
15                    if len(other_cast) > max_shared_cast:
16
17                        shared_cast = len(cast.intersection(other_cast))
18
19                        if shared_cast > max_shared_cast:
20                            max_shared_cast = shared_cast
21                            movie_pair = (movie_list[i], movie_list[j])
22
23                else: # if j-th movie's cast is too small
24                    discarded.add(movie_list[j])
25
26            else: # if i-th movie's cast is too small
27                discarded.add(movie_list[i])
```

Results

Movie	Shared Cast
Kingdom Hearts II (2005) (VG)	194
Kingdom Hearts II: Final Mix+ (2007) (VG)	

Goal

Build actor graph, whose nodes are only actors and two actors are connected if they did a movie together.

Find the actors who collaborated the most among themselves.

Algorithm

The algorithm proposed creates a weighted graph where the weight of an edge represents the times two actors have worked together.

Algorithm

The algorithm proposed creates a weighted graph where the weight of an edge represents the times two actors have worked together.

The edge with highest weight connects the actors who worked together the most.

Algorithm

To build the graph, the algorithm relies on the previous actor-movie graph.

Algorithm

To build the graph, the algorithm relies on the previous actor-movie graph.

For each movie and its cast, consider all possible combinations of pairs of actors:

- if actors' nodes already share an edge, increase its weight
- if not, create an edge with unitary weight

```
1 # create a node for every actor
2 actors_graph.add_nodes_from(act_dict.values())
3
4 # iterate over movies in the graph
5 for movie_id in movie_dict.values():
6     cast = list(graph.adj[movie_id])
7
8     if len(cast) > 1:
9         # consider all possible combinations
10        for pair in itertools.combinations(cast, 2):
11
12            if actors_graph.has_edge(pair[0], pair[1]):
13                # increase edge weight
14                actors_graph.edges[pair[0], pair[1]]["weight"] += 1
15
16            else:
17                # add edge with unitary weight
18                actors_graph.add_edge(pair[0], pair[1], weight=1)
19
20 return actors_graph
```

Results

The computer incurred in a memory error before creating the graph.